



SAKARYA
ÜNİVERSİTESİ

Bilgisayar ve Bilişim Bilimleri Fakültesi
Bilgisayar Mühendisliği Bölümü

Sistem Programlama Dersi Proje Ödevi

Hazırlayanlar:

Koray Temizkan: B231210010 / 1-A

Furkan Bağcı: B231210090 / 1-A

Bağlantılar:

Github Reposu: <https://github.com/KorayTemizkan/sistem-programlama-odev>

Proje Özeti:

Bu raporda, Sistem Programlama dersi kapsamında geliştirilen "tarsau" adlı arşivleme programının tasarım, geliştirme ve test süreçleri sunulmaktadır. Projenin temel amacı; Linux/Unix tabanlı işletim sistemlerinde çalışan, tar veya zip gibi endüstri standartlarındaki araçların temel arşivleme mantığını taklit eden ancak sıkıştırma yapmayan, C dili ile yazılmış bir komut satırı aracı geliştirmektir.

Geliştirilen "tarsau" programı iki temel işlevi yerine getirmektedir:

- 1) Arşivleme (Birleştirme -b): Kullanıcı tarafından belirtilen, tamamen ASCII karakterlerden oluşan maksimum 32 adet metin dosyasını (toplam boyutu 200 MB'ı aşmayacak şekilde) tek bir .sau uzantılı arşiv dosyasında birleştirir. Bu işlem sırasında dosyaların isimleri, boyutları ve sistem izinleri (okuma, yazma, çalıştırma) özel bir başlık (header) formatında arşivin en başına şifrelenerek kaydedilmektedir.
- 2) Arşivden Çıkarma (Açma -a): Oluşturulan .sau uzantılı arşiv dosyasını ayrıştırarak (parsing), içerisindeki metin dosyalarını belirtilen hedef dizine (dizin yoksa yeni oluşturularak) çıkartır. Çıkarma işlemindeki en kritik adım olan "orijinal dosya izinlerinin korunması" gereksinimi, POSIX sistem çağrılarını kullanarak eksiksiz bir şekilde sağlanmıştır.

Proje süreci boyunca C dilinde düşük seviyeli dosya giriş/çıkış işlemleri (File I/O), dinamik bellek yönetimi (malloc, free), karakter katarı ayrıştırma (strtok, sscanf) ve Linux sistem çağrıları (stat, chmod, mkdir) yoğun olarak kullanılmıştır. Ayrıca, projenin geliştirme adımları ve kaynak kod yönetimi, ekip çalışmasına uygun olarak GitHub üzerinden versiyon kontrol sistemi ile yürütülmüştür.

Sonuç olarak; istenen tüm teknik parametreleri karşılayan, hatalı kullanıcı girişlerine karşı güvenli (exception handling) ve bellek sızıntısı bulundurmeyen stabil bir sistem aracı başarıyla ortaya konmuştur.

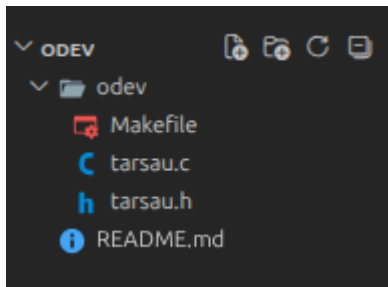
Önemli Notlar:

Bu proje, Koray Temizkan'ın "Linux Mint" işletim sistemi kurulu bilgisayarındaki "Visual Studio Code" editöründe test edilmiş ve sorunsuz çalışmıştır.

Bu proje, Furkan Bağcı'nın "Windows" işletim sistemi kurulu bilgisayarındaki "WSL" yardımıyla test edilmiş ve sorunsuz çalışmıştır.

Dizin ve Kod Yapıları:

Dizin Yapısı:



Başlık Dosyası:

```
h tarsau.h X
odev > h tarsau.h > ...
1  #ifndef TARSAU_H
2  #define TARSAU_H
3
4  // String Kütüphaneleri
5  #include <stdio.h>
6  #include <stdlib.h>
7  #include <string.h>
8  #include <sys/types.h> // İşletim sistemi veri tipleri (mode_t vb.)
9  #include <sys/stat.h>  // Dosya durumu ve izinleri okumak için (stat fonksiyonu)
10 #include <unistd.h>    // POSIX işletim sistemi çağrıları (okuma/yazma erişimleri)
11
12 // Proje isterlerine göre belirlenen kısıtlar
13 #define MAX_FILES 32          // Maksimum giriş dosyası sayısı
14 #define MAX_TOTAL_SIZE 200 * 1024 * 1024 // Maksimum toplam boyut (200 MB)
15
16 // Dosya başlık (header) bilgilerini tutmak için yapı
17 typedef struct
18 {
19     char filename[256];
20     mode_t permissions;
21     long size;
22 } FileRecord;
23
24 #endif
```

Kaynak Dosyasının Örnek Olarak Giriş Bölümü:

```
tarsau.c x h tarsau.h
odev > C tarsau.c > main(int, char *[])
1  #include "tarsau.h"
2
3  /*
4   * Parametre olarak verilen dosyanın içeriğini okuyarak
5   * sadece ASCII karakterlerden oluşup oluşmadığını denetler.
6   * ASCII dışı bir karakter (değeri 0'dan küçük veya 127'den büyük)
7   * bulunursa 0, tamamen ASCII ise 1 döndürür.
8   */
9
10 int is_ascii(const char *filename)
11 {
12     FILE *f = fopen(filename, "r");
13     if (!f)
14         return 0; // Dosya açılamazsa güvenli çıkış yap
15
16     int c;
17     // Dosya sonuna (EOF) gelene kadar bayt bayt oku
18     while ((c = fgetc(f)) != EOF)
19     {
20         if (c < 0 || c > 127)
21         {
22             fclose(f);
23             return 0; // Uyumsuz karakter bulundu
24         }
25     }
26     fclose(f);
27     return 1; // Dosya formata uygun
28 }
29
30 int main(int argc, char *argv[])
31 {
32     // Yetersiz argüman kontrolü
33     if (argc < 2)
34     {
35         printf("Kullanım: ./tarsau -b [dosyalar] -o [cikti.sau] veya ./tarsau -a [arsiv.sau] [dizin]\n");
36         return 1;
37     }
38
39     // BİRLEŞTİRME (ARŞİVLEME) MODU (-b)
40     // Birden fazla metin dosyasını sıkıştırma yapmadan tek dosyada toplar
41     if (strcmp(argv[1], "-b") == 0)
42     {
43         char *output_name = "a.sau"; // Kullanıcı isim vermezse varsayılan arşiv adı
44         int file_count = 0;
45         char *input_files[MAX_FILES];
46         long total_size = 0;
47
48         // Komut satırı argümanlarını ayrıştırma
49         for (int i = 2; i < argc; i++)
50         {
51             if (strcmp(argv[i], "-o") == 0)
52             {
53                 // -o parametresi geldiyse, bir sonraki argümanı çıktı adı olarak al
54                 if (i + 1 < argc)
55                     output_name = argv[++i];
56             }
57             else
58             {
59                 // Girilen dosya sayısı sınırı aşıyor mu kontrol et
60                 if (file_count >= MAX_FILES)
61                 {
62                     printf("Hata: En fazla 32 giriş dosyası belirtilebilir!\n");
63                     return 1;
64                 }
65                 input_files[file_count++] = argv[i];
66             }
67         }
68     }
```

Terminal Çalıştırma Adımları:

1. Derleme:

```
make clean
```

```
make
```

2. Test Dosyası Hazırlama:

```
echo "test yazisi" > t1.txt
```

```
echo "test yazisi 2" > t2.txt
```

```
chmod 777 t1.txt
```

3. Arşivleme (Birleştirme):

```
./tarsau -b t1.txt t2.txt -o test.sau
```

4. Arşivden Çıkarma:

```
./tarsau -a test.sau cikti_klasoru
```

5. İzin ve Sonuç Kontrolü:

```
ls -l cikti_klasoru
```

Görsel Terminal Çıktıları:

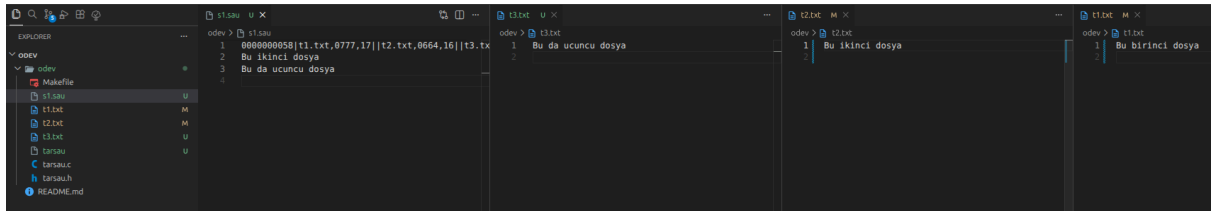
Git bağlantısı kontrolü -> make ile derleniyor -> txt dosyaları oluşturuldu.

```
▼ TERMINAL
• koray@KoraysLinuxPC:~/Ders/3_2/yeni-sistem-programlama/odev/odev$ git status
On branch main
Your branch is up-to-date with 'origin/main'.

nothing to commit, working tree clean
• koray@KoraysLinuxPC:~/Ders/3_2/yeni-sistem-programlama/odev/odev$ make clean
rm -f tarsau
• koray@KoraysLinuxPC:~/Ders/3_2/yeni-sistem-programlama/odev/odev$ make
gcc tarsau.c -o tarsau
• koray@KoraysLinuxPC:~/Ders/3_2/yeni-sistem-programlama/odev/odev$ echo "Bu birinci dosya" > t1.txt
echo "Bu ikinci dosya" > t2.txt
echo "Bu da ucuncu dosya" > t3.txt
chmod 777 t1.txt
ls -l t*.txt
-rwxrwxrwx 1 koray koray 17 May 13 17:39 t1.txt
-rw-rw-r-- 1 koray koray 16 May 13 17:39 t2.txt
-rw-rw-r-- 1 koray koray 19 May 13 17:39 t3.txt
```

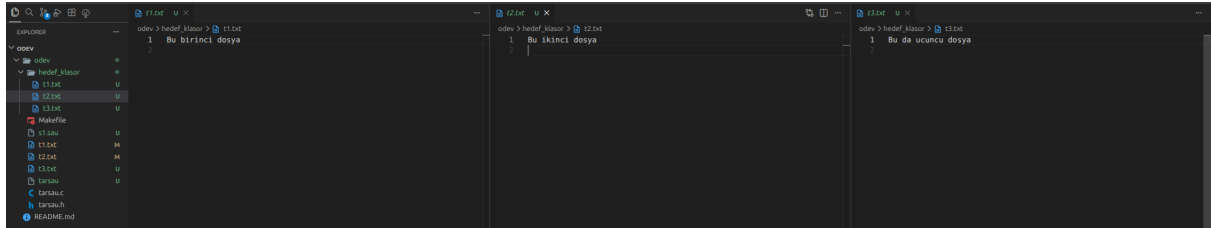
Dosyalar birleştirildi.

```
• koray@KoraysLinuxPC:~/Ders/3_2/yeni-sistem-programlama/odev/odev$ ./tarsau -b t1.txt t2.txt t3.txt -o s1.sau
Dosyalar birleştirildi.
```



Dosyalar açıldı.

```
• koray@KoraysLinuxPC:~/Ders/3_2/yeni-sistem-programlama/odev/odev$ ./tarsau -a s1.sau hedef_klasor
ls -l hedef_klasor
hedef_klasor dizininde t1.txt, t2.txt, t3.txt dosyaları açıldı.
total 12
-rwxrwxrwx 1 koray koray 17 May 13 17:41 t1.txt
-rw-rw-r-- 1 koray koray 16 May 13 17:41 t2.txt
-rw-rw-r-- 1 koray koray 19 May 13 17:41 t3.txt
```



Hata kontrolü.

```
• koray@KoraysLinuxPC:~/Ders/3_2/yeni-sistem-programlama/odev/odev$ ./tarsau -b t1.txt tarsau -o s2.sau
tarsau giriş dosyasının formatı uyumsuzdur!
• koray@KoraysLinuxPC:~/Ders/3_2/yeni-sistem-programlama/odev/odev$ ./tarsau -a t1.txt yeni_dizin
Arşiv dosyası uygunsuz veya bozuk!
```